CS 316 Project 4: Multithreaded Monte Carlo Simulation for Estimating π

Learning objectives:

1. Apply Java's Executor framework to implement multithreading.
2. Compare single-threading and multi-threading in terms of performance.
3. Describe the basic concept of the Monte Carlo method.

What to submit:

1. Submit the individual Java source files that implement the multithreaded version.
2. Complete and submit the following table in a separate **PDF** file, using the π values and runtimes produced by the single-threaded and multi-threaded Java implementations. (Simulating 10 billion points may take a few minutes to complete, depending on your hardware.)

	Single-threaded runtime	Multi-threaded runtime (4 threads)	Single-threaded π value	Multi-threaded π value
1 million total points				
100 million total points				
10 billion total points				

Note: please submit the files individually, and do not submit any other files.

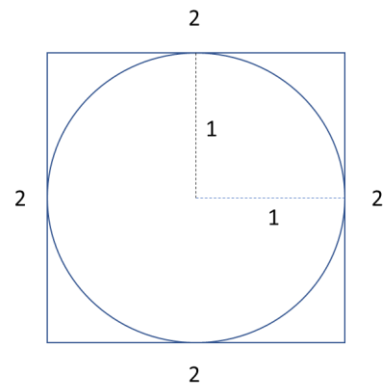
Background:

The Monte Carlo method uses random sampling to estimate values that are difficult or expensive to compute directly. Accuracy improves with the number of samples used, but large-scale simulations can be slow. Multithreading allows us to perform multiple simulations in parallel, improving performance by using multiple CPU cores.

This technique is widely used in fields like physics, mathematics, engineering, and finance. To learn more, refer to the Wikipedia article on Monte Carlo methods.

Instructions:

1. In this project, we use the Monte Carlo method to approximate the value of π. Here's the idea: imagine a circle of radius 1. The area of the circle is $\pi * 1^2$, which is just π. This circle is tightly enclosed by a 2 by 2 square, whose area is 4.



We *randomly* generate points inside the square. These points may appear anywhere within the square, and a portion of them will be inside the circle. The ratio of points inside the circle to total points approximates the ratio of the two areas:

$$\frac{area\ of\ the\ circle}{area\ of\ the\ square} = \frac{number\ of\ points\ inside\ circle}{total\ number\ of\ points}$$

Plugging in the area values, we have:

$$\frac{\pi}{4} = \frac{number\ of\ points\ inside\ circle}{total\ number\ of\ points}$$

So, π can be calculated as:

$$\pi = 4 * \frac{number\ of\ points\ inside\ circle}{total\ number\ of\ points}$$

This is, of course, only an approximation. Its accuracy improves as the number of samples increases—that is, as we generate more random points.

2. We've provided a single-threaded implementation in **SingleThreadedSimulation.java**, which performs the entire simulation in the main thread. It also records the total runtime. Please review this file carefully as you'll be able to reuse much of its code when implementing the multithreaded version (described below).

3. You job is to implement the multi-threaded version. More specifically:

    a. Use four threads to perform the simulation in parallel, so your implementation should create and submit four simulation tasks.

    b. Each task should generate $\frac{1}{4}$ of the total points.

    c. Each task should return the number of points that fall inside the circle. Importantly, do *not* calculate the π value inside any individual task—it should be computed once in the main thread after all tasks have completed.

    d. The main thread must perform the follow steps in the order described:

        i. Submit all four tasks using an **ExecutorService**

        ii. Collect the results from the tasks

iii. Sum the counts and compute the final π estimate using the formula described above

iv. Record the total runtime

4. Please note that your implementation must use the *Java Executor framework*, as it is a core learning objective of this project.

Grading:

This project is worth a total of 100 points, distributed as follows.

1. Implement the simulation task: 30 points
2. Create the **ExecutorService** object representing a thread pool: 5 points
3. Create and submit four simulation tasks: 10 points
4. Collect the simulation results and calculating the π value in the main thread: 15 points
5. Completing the table based on your simulations: 40 points

Please note: you must use the *Java Executor framework* for the multi-threaded implementation. If your implementation is done in any other way, you will not receive full credit for that part of the project.

Additional Notes:

Java provides multiple ways to generate random numbers. In a multi-threaded program, it is recommended that we use the **java.util.concurrent.ThreadLocalRandom** class, as it is thread-safe and each thread gets its own internally generated seed. Use of **ThreadLocalRandom** is especially appropriate when multiple tasks generate random numbers in parallel in a thread pool.

For example, to generate a random floating-point number between -1 and 1, we can do:

double x = ThreadLocalRandom.current().nextDouble(-1, 1);

For more information about this class, please refer to the **ThreadLocalRandom** Javadoc.